NIET Greater Noida
Autonomous Institute
Get Future Ready

Noida Institute of Engineering and Technology, Greater Noida

Operating Systems
(-------)

Unit: 2

Process Management &
Thread Management

B Tech 3rd Sem

Ms. Vaishali Mishra
Assistant Professor
Department of CSE

Subject Syllabus

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY
GREATER NOIDA-201306
(An Autonomous Institute)
School of Computer Science & Information Technology

Subject Name: Operating Systems	**L-T-P [2-0-0]**
Subject Code:	**Applicable in Department: CSE/CSE-R/IT/CS/AI/AIML/ IOT/DS/CYS**
Pre-requisites of the Subject: Basic understanding of C/ C++ Programming, Data structures & algorithms, Computer Organization & architectures.	
Course Objective-The objective of the course is to provide a foundational understanding of operating system concepts, including system architecture, process and thread management, concurrency, deadlock, resource management, memory and file systems, Linux shell scripting, and an introduction to virtualization and distributed systems.	

Course Outcomes (CO)

	Course outcome: After completion of this course, students will be able to:	**Bloom's Knowledge Level(KL)**
CO 1	Understand operating systems architecture and types, and use the Linux CLI for basic Operations.	K2
CO2	Implement the CPU scheduling algorithms along with uses of multithreading models.	K4
CO3	Implement concurrency control, process synchronization techniques, and deadlock handling methods.	K4
CO4	Implement memory management strategies and page replacement algorithms to optimize system performance.	K4
CO5	Analyze file systems and configure distributed systems and virtual machines in modern operating systems.	K4

NIET Greater Noida
Autonomous Institute
Get Future Ready

Subject Syllabus

Syllabus						
Unit No	Module Name	Topic covered	Pedagogy	Lecture Required (L+P)	Practical/ Assignment/ Lab Nos	CO Mapping
1 Fundamentals & Shell scripting	Module 1.1: Fundamentals of Operating Systems	Fundamentals of Operating Systems, Overview of Operating Systems, Generations of OS, Operating system architecture, Interrupt handling, System call and kernel, Types of Operating System: Batch OS, Multiprogramming OS, Multitasking OS, Multiprocessor OS, Real time OS.	Class room Teaching, Smart Board, PPT.	2	Assignment 1	CO1
	Module 1.2: Shell Scripting in Linux	Shell Scripting in Linux, Introduction to Linux Operating System & Architecture, Basic Command Line Interface (CLI) Operations in Linux, Shell Scripting Basics: Variables, Control Structures, Functions Case Study: Automating system administration tasks using shell scripts in Ubuntu/Linux (e.g., backup scheduling).	Class room Teaching, Smart Board, PPT.	2		CO1
2 Process & Thread Management	Module 2.1: Process Management and CPU Scheduling	**Process Management**: - Process, Transition Diagram, Process Control Block (PCB), Types of Schedulers: Long Term, Mid Term, Short Term Scheduler. **CPU Scheduling**- Pre-emptive and Non-Pre-emptive Algorithm (FCFS, SJF, SRTF, Non-Pre-emptive Priority, Pre-emptive Priority, Round Robin, Multilevel Queue Scheduling and Multilevel Feedback Queue Scheduling) .	Class room Teaching, Smart Board, PPT.	4	Practical 4 to 9	CO2
	Module 2.2: Thread	Thread: - Processes vs Threads, Thread states, Benefits of threads, Types of threads, Multithread Model, Concept of Hyper-Threading Case Study: - Analyse and implement CPU Scheduling in Real-Time Embedded Systems and RTOS	Class room Teaching, Smart Board, PPT.	4	Assignment 1	CO2

NIET Greater Noida
Autonomous Institute
Get Future Ready

Subject Syllabus

3 Concurrency and Deadlock Management	Module 3.1: Concurrency, Process Synchronization	Concurrency: Introduction of Concurrency, Types of Process, Race Condition, Critical Section, Inter Process Communication, Producer consumer problem. Process Synchronization: Lock variable, Peterson's Solution, Strict alternation, Lamport Bakery Solution, Test and set lock, Semaphore- counting, binary and monitor, Classical Problem of Synchronization: - Bound Buffer, Dinning Philosopher, Reader writer, Sleeping barber.	Class room Teaching, Smart Board, PPT.	4	Assignment 2, Practical 10,11,12	CO3
	Module 3.2: Deadlock	Deadlock: Deadlock, Deadlock characterization, Deadlock Prevention, Deadlock Avoidance: Bankers Algorithms, Deadlock Detection, Recovery from Deadlock. Case Study: Deadlock avoidance in database transaction management systems like Oracle or MySQL.	Class room Teaching, Smart Board, PPT.	4	Practical 13,14	CO3

4 Memory Management	Module 4.1: Memory Management	Memory Management: - Memory Management function, Loading and linking Address Binding, Memory management techniques, Contiguous technique- Fixed Partitions, variable partitions, Memory Allocation: Allocation Strategies (First Fit, Best Fit, and Worst Fit), Non-contiguous, Paging, Segmentation, Segmented paging	Class room Teaching, Smart Board, PPT.	4	Assignment 3	CO4
	Module 4.2: Virtual memory	Virtual Memory:- Virtual Memory Concepts, Demand Paging, Performance of Demand Paging, Page Replacement Algorithms: FIFO, LRU, Optimal and LFU, Belady's Anomaly, Thrashing Case Study: Virtual memory management in modern OS like Windows 10 and how paging impacts performance.	Class room Teaching, Smart Board, PPT.	4	Practical 21 to 23	CO4

NIET Greater Noida
Autonomous Institute
Get Future Ready

Subject Syllabus

5 File Management & Modern Operating System	Module 5.1:	File Management: - Access Mechanism, File Allocation Method, Free Space Management: -Bit Vector, Linked List. DISK: Disk Architecture, HDD vs SSD, Disk Scheduling Algorithms	Class room Teaching, Smart Board, PPT.	2	Practical 22 to 24	CO5
	Module 5.2:	Modern Operating System: -Overview of modern operating system, Modern OS features: Multitasking, virtualization, security, scalability, Shared Memory concepts, Distributed system, Parallel system & its architecture, Virtual machines – hypervisor, Introduction to GPU Case Study: Large File Storage in a Distributed Manner.	Class room Teaching, Smart Board, PPT.	2	Assignment 3	CO5

Textbooks	
Sr No	**Book Details**
1.	Abraham Silberschatz, Peter Baer Galvin and Greg Gagne" Operating System Concepts Essentials" , Willey Publication,8th Edition,2017.
2.	Marks G. Sobell "A practical guide to Linux: Commands, Editors and Shell Programming", CreateSpace Independent Publishing Platform, 4th Edition,2017.
3.	Jason Cannon "LINUX for beginners", 1stEdtion,2014
Reference Books	
Sr. No.	**Book Details**
1.	William Stallings "Operating Systems: Internals and Design Principles", Pearson Education , 9th Edition, 2019.
2.	Charles Patrick Crowley, "Operating System: A Design-oriented Approach" , McGraw Hill Education ,2017,

NIET Greater Noida
Autonomous Institute
Get Future Ready

Branch wise Applications

- Airlines reservation system.
- Air traffic control system.
- Systems that provide immediate updating.
- Used in any system that provides up to date and minute information on stock prices.
- Defense application systems like RADAR.
- Networked Multimedia Systems.
- Command Control Systems.

NIET
Greater Noida
Autonomous Institute
Get Future Ready

Prerequisite

- Basic knowledge of computer fundamentals.
- Basic knowledge of computer organization, Memory hierarchy, Cache Organization, Interrupt, Registers, Associative memory
- Basic Operating System Concepts .
- Data Structures – Queues, linked lists, and priority queues for process scheduling. Understanding different scheduling techniques and their efficiency.
- Linux Process Management .
- Familiarity with process-related commands in Linux.

NIET
Greater Noida
Autonomous Institute
Get Future Ready

Recap

- **Operating System Basics**
  Covered the **definition, purpose, and architecture** of an Operating System (OS), including **interrupt handling**, **system calls**, and the **kernel**.
- **Evolution of Operating Systems**
  Explored the **generations of OS**, showing how they evolved from **Batch Systems to Real-Time Operating Systems**, including **Multiprogramming**, **Multitasking**, and **Multiprocessor OS** types.
- **Linux Operating System & Architecture**
  Introduced the **Linux OS structure**, its **kernel-user space** design, and the use of **command-line interfaces** (CLI) for basic operations.
- **Shell Scripting Essentials**
  Learned the **basics of shell scripting**: declaring **variables**, using **control structures** (if, loops), and writing **functions** to automate tasks.
- **Practical Application through Case Study**
  Demonstrated **system administration automation** (e.g., **backup scheduling**) using shell scripts in Ubuntu/Linux—laying the foundation for scripting in real environments.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Unit-2 Content

- Process
- Process Performance Criteria,
- Process Transition Diagram,
- Process Control Block (PCB),
- Types of Schedulers: Long Term, Mid Term, Short Term Scheduler,
- CPU Scheduling- Pre-emptive and Non-Pre-emptive Algorithm (FCFS, SJF, SRTF, Non-Pre-emptive Priority, Pre-emptive Priority, Round Robin, Multilevel Queue Scheduling and Multilevel Feedback Queue Scheduling),
- Processes and Threads, Thread states, Benefits of threads, Types of threads, Multithread Model, Concept of Hyper-Threading
- Case Study: - Analyse and implement CPU Scheduling in Real-Time Embedded Systems and RTOS

NIET Greater Noida
Autonomous Institute
Get Future Ready

Introduction

Process management involves various tasks like creation, scheduling, termination of processes, and a deadlock. The process is a program that is under execution, which is an important part of modern-day operating systems. The OS must allocate resources that enable processes to share and exchange information. It also protects the resources of each process from other methods and allows synchronization among processes.

The scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process based on a particular strategy.

For video lecture: https://www.youtube.com/watch?v=zWtFoPL8BYg

Topic1

Process

Objective:

Students will understand the concept of a process, how it differs from a program, and the importance of process management in multitasking operating systems.

Prerequisite:

- Understanding of OS basics: memory, CPU, program execution.
- Familiarity with OS architecture and system calls (from Unit 1).

Introduction (Program Vs. Process)

Program

- It is a set of instructions that has been designed to complete a certain task.
- It is a passive entity.
- It resides in the secondary memory of the system.
- It exists in a single place and continues to exist until it has been explicitly deleted.
- It is considered as a static entity.
- It doesn't have a resource requirement.
- It requires memory space to store instructions.
- It doesn't have a control block.

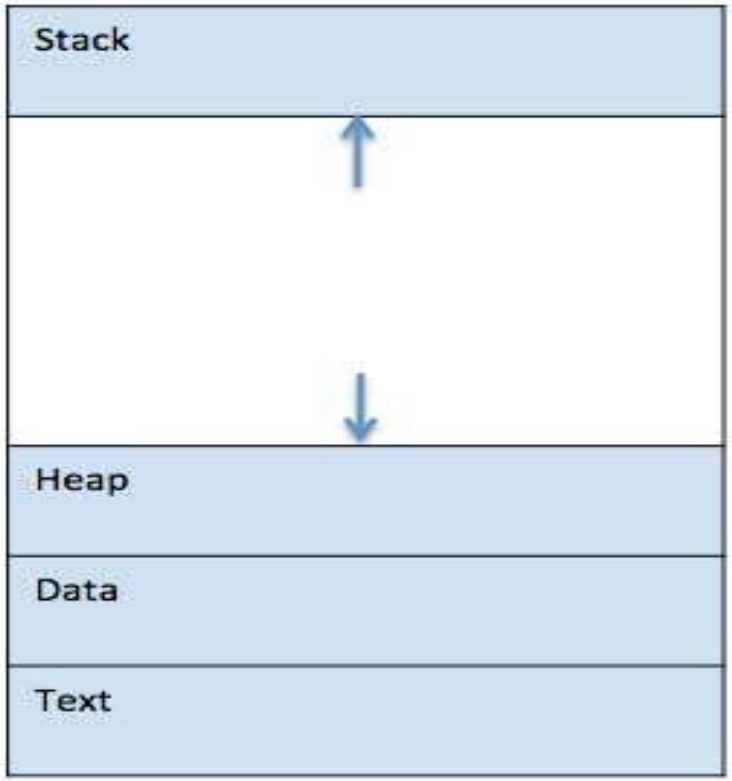
Introduction (Program Vs. Process)

Process

- It is an instance of a program that is being currently executed.
- It is an active entity.
- It is created when program is in execution and is being loaded into the main memory.
- It exists for a limited amount of time.
- It gets terminated once the task has been completed.
- It is considered as a dynamic entity.
- It has a high resource requirement.
- It requires resources such as CPU, memory address, I/O during its working.
- It has its own control block, which is known as Process Control Block.

Process

- A process is basically a program in execution. The execution of a process must progress in a sequential fashion.
- A process is defined as an entity which represents the basic unit of work to be implemented in the system.
- To put it in simple terms, we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.
- When a program is loaded into the memory and it becomes a process, it can be divided into four sections – stack, heap, text and data. The following image shows a simplified layout of a process inside main memory



Process (Recap)

- **Recap:**
  - A process is a program in execution with its own memory, state, and control data.
  - Differentiates from a program (which is passive).

Process (MCQ)

Select the term that refers to a running program with allocated resources.

A) Source code
B) A running instance of a program *
C) A compiled binary
D) A file in memory

Identify the key element that distinguishes a process from a program.

A) Syntax used
B) Execution and resource context*
C) Program name
D) File size

Move a process from “waiting” to “ready” state after this condition is met.

A) Time slice ends
B) I/O request completes*
C) Process is created
D) Program crashes

Topic 2 (Objective & Prerequisite)

Process Performance Criteria

Objective:

Students will be able to evaluate and compare different scheduling algorithms using key performance metrics like turnaround time, waiting time, response time, throughput, and CPU utilization.

Prerequisite:

- Basic math and logic (e.g. turnaround time, waiting time).

NIET Greater Noida
Autonomous Institute
Get Future Ready

Process Performance Criteria

Various criteria or characteristics that help in designing a good scheduling algorithm are:

Key Metrics:

- **CPU Utilization**: Percentage of time the CPU is busy. A scheduling algorithm should be designed so that CPU remains busy as possible. It should make efficient use of CPU.
- **Throughput**: Number of processes or work completed per unit time.
- **Turnaround Time**: Total time taken from submission to completion. Thus how long it takes to execute a process is also an important factor.
- **Waiting Time**: Time spent waiting in the ready queue. It is the time a job waits for resource allocation when several jobs are competing in multiprogramming system. The aim is to minimize the waiting time.
- **Response Time**: Time from submission to the first response (output).

Process Performance Criteria (Recap)

- **Recap:**
  - Key metrics: CPU utilization, throughput, turnaround time, waiting time, response time.
  - These help evaluate scheduling algorithms.

NIET Greater Noida Autonomous Institute

Process Performance Criteria (MCQ)

- Higher CPU utilization leads to this system outcome.
  A) More idle time
  B) Better performance*
  C) Greater memory load
  D) Fewer I/O requests

- CPU burst time equals turnaround time minus this delay.
  A) Response time
  B) Execution time
  C) Waiting time*
  D) Arrival time

- Throughput for 100 processes completed in 5 seconds.
  A) 20/sec *
  B) 50/sec
  C) 5/sec
  D) 25/sec

Topic 3 (Objective & Prerequisite)

Process Transition Diagram

Objective:

Students will learn to describe and explain various states of a process and transitions between them using the process state diagram.

Prerequisite:

- Concepts of process states and OS process lifecycle.

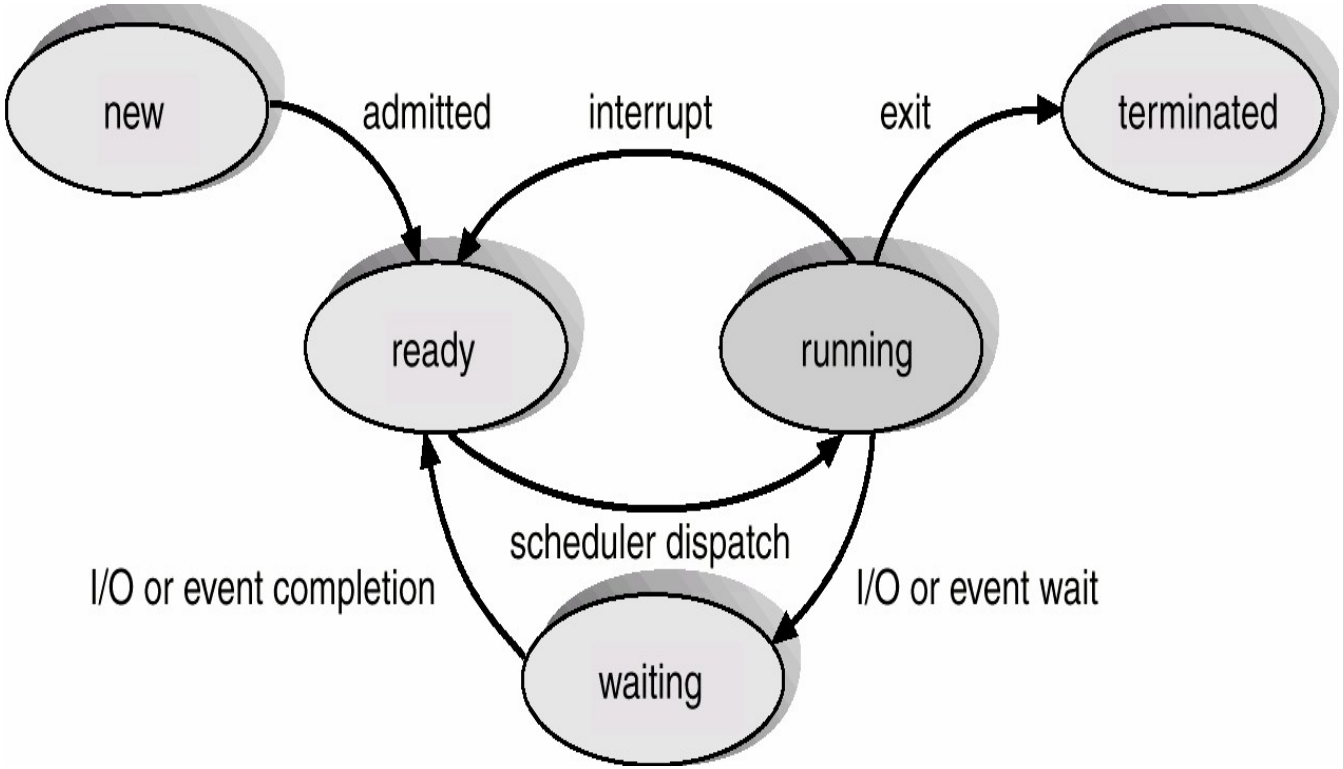
NIET Greater Noida Autonomous Institute

Process Transition Diagram

States of a Process:

1. **New**: Process is being created.
2. **Ready**: Process is ready to execute but waiting for CPU.
3. **Running**: Process is currently executing.
4. **Waiting**: Process is waiting for an I/O operation.
5. **Terminated**: Process has completed execution.

Process Transition Diagram



Process Transition Diagram (Recap)

Recap:

Shows states like New, Ready, Running, Waiting, Terminated.
Transitions depend on scheduling, I/O, and events.
Important for understanding process control.

Process Transition Diagram (MCQ)

CPU preemption triggers transition to this state.
A) Waiting
B) Terminated
C) Ready*
D) New
A process moves to this state after I/O completion.
A) Running
B) Terminated
C) Ready*
D) Waiting
Task completion leads the process to this final state.
A) Waiting
B) Ready
C) Terminated *
D) Suspended

Topic 4 (Objective & Prerequisite)

Process Control Block (PCB)

Objective:

Students will understand the role of the Process Control Block in process management and explain its components used for context switching and process control.

Prerequisite:

Awareness of what a process needs to execute (memory, ID, status).

NIET Greater Noida
Autonomous Institute
Get Future Ready

Process Control Block (PCB)

Information associated with each process:

- **Process ID (PID)**: Unique identifier for each process.
- **Process State**: Current state (e.g., ready, running).
- **Program Counter**: contains Address of the next instruction that needs to be executed in the process.
- **CPU Registers**: Contents of all processor registers.
- **CPU scheduling information** The process priority, pointers to scheduling queues etc.
- **Memory Management Info**: Base and limit registers, page tables.
- **I/O Information**: Open files and I/O devices allocated.
- **Accounting Information**: CPU usage, time limits, and process priority.

Process Control Block (PCB)

PCB Components:

Process State

This specifies the process state i.e. new, ready, running, waiting or terminated.

Process Number

This shows the number of the particular process.

Program Counter

This contains the address of the next instruction that needs to be executed in the process.

Registers

This specifies the registers that are used by the process. They may include accumulators, index registers, stack pointers, general purpose registers etc.

List of Open Files

These are the different files that are associated with the process

Process Control Block (PCB)

PCB Components:

CPU Scheduling Information

The process priority, pointers to scheduling queues etc. is the CPU scheduling information that is contained in the PCB. This may also include any other scheduling parameters.

Memory Management Information

The memory management information includes the page tables or the segment tables depending on the memory system used. It also contains the value of the base registers, limit registers etc.

I/O Status Information

This information includes the list of I/O devices used by the process, the list of files etc.

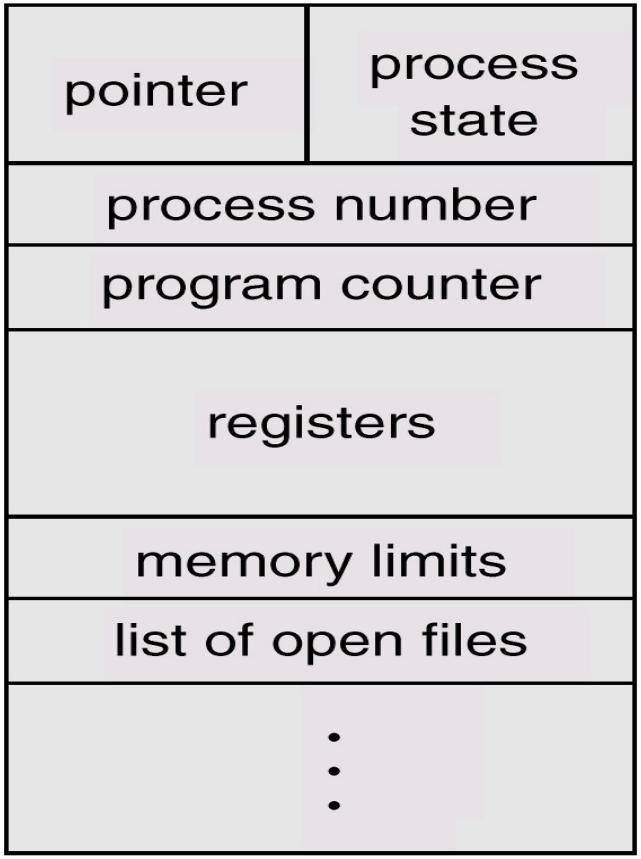
Accounting information

The time limits, account numbers, amount of CPU used, process numbers etc. are all a part of the PCB accounting information.

NIET Greater Noida Autonomous Institute

Get Future Ready

Process Control Block (PCB)



Process Control Block (PCB)(Recap)

Recap:

PCB stores all essential info of a process: PID, state, registers, scheduling info.
It is used by OS for context switching.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Process Control Block (PCB)(MCQ)

Purpose of PCB in managing running processes.
A) Resource scheduling
B) Process switching*
C) Device control
D) File encryption
OS stores this structure during context switching.
A) Thread ID
B) PCB*
C) Stack frame
D) I/O table
Suspended process data saved in this location.
A) Stack
B) Ready queue
C) PCB*
D) RAM

Topic 5 (Objective & Prerequisite)

Types of Schedulers

Objective:

Students will be able to distinguish between the three types of schedulers, explain their responsibilities, and understand their impact on system performance and resource management.

Prerequisite:

- Concept of Ready Queue and process states.

Types of Schedulers

1. **Long-Term Scheduler**:
    1. Decides which processes are admitted to the system.
    2. Controls degree of multiprogramming.
    3. Runs less frequently.
2. **Mid-Term Scheduler**:
    1. Swaps processes in/out of memory.
    2. Manages partially executed processes.
    3. Aims to optimize memory usage.
3. **Short-Term Scheduler**:
    1. Selects a process from the ready queue for execution.
    2. Runs frequently and directly affects performance.

Types of Schedulers (Recap)

Recap:

- **Long-term Scheduler**: selects which processes to admit.
- **Short-term Scheduler**: selects which process runs next.
- **Mid-term Scheduler**: handles suspended processes.

Types of Schedulers (MCQ)

•Scheduler that decides when new jobs enter the system.
A) Long-term scheduler*
B) Short-term scheduler
C) Mid-term scheduler
D) I/O scheduler
•Scheduler that runs most frequently.
A) Long-term
B) Disk
C) Short-term*
D) Boot
•Mid-term scheduler performs this main function.
A) Device scanning
B) Load balancing
C) Queue monitoring
D) Swapping suspended processes*

Topic 6 (Objective & Prerequisite)

CPU Scheduling Algorithms (Preemptive & Non-Preemptive)

Objective:

Students will learn and implement various CPU scheduling algorithms (FCFS, SJF, SRTF, Priority, Round Robin, etc.) and compare their performance under different workloads.

Prerequisite:

- Concept of process queue and time-sharing.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Process Scheduling Queues

- Job queue – set of all processes in the system.
- Ready queue – set of all processes residing in main memory, ready and waiting to execute.
- Device queues – set of processes waiting for an I/O device.
- Processes migrate between the various queues

CPU Scheduling Algorithms

Types:

A. Pre-emptive Scheduling:

- Allows processes to be interrupted and moved back to the ready queue.
- Examples: Round Robin, SRTF, Pre-emptive Priority.

B. Non-Pre-emptive Scheduling:

- A process runs to completion once it starts.
- Examples: FCFS, SJF, Non-Pre-emptive Priority.

NIET Greater Noida Autonomous Institute

Get Future Ready

First-Come, First-Served (FCFS):

- In FCFS, the process which arrives first in front of CPU will be executed first by the CPU.
- It is a non-preemptive type of scheduling algorithm, i.e. in this scheduling algorithm priority of processes does not matter, or whatever the priority of the process is, the process will be executed in the manner they arrived in front of the CPU.

First-Come, First-Served (FCFS):

Process	Burst Time
P_1	24
P_2	3
P_3	3

Suppose that the processes arrive in the order: P_1 , P_2 , P_3.
The Gantt Chart for the schedule is:

Waiting time for P_1 = 0; P_2 = 24; P_3 = 27
Average waiting time: (0 + 24 + 27)/3 = 17

First-Come, First-Served (FCFS):

Suppose that the processes arrive in the order
P_2 , P_3 , P_1.

The Gantt chart for the schedule is:

Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$

Average waiting time: (6 + 0 + 3)/3 = 3

Much better than previous case.

Convoy effect short process behind long process

First-Come, First-Served (FCFS):

Characteristics of FCFS method:

- It offers non-preemptive and pre-emptive scheduling algorithm.
- Jobs are always executed on a first-come, first-serve basis
- It is easy to implement and use.
- However, this method is poor in performance, and the general wait time is quite high.

NIET
Greater Noida
Autonomous Institute
Get Future Ready

Problems with FCFS Scheduling

Below we have a few shortcomings or problems with the FCFS scheduling algorithm:

- It is **Non Pre-emptive** algorithm, which means the **process priority** doesn't matter.If a process with very least priority is being executed, more like **daily routine backup** process, which takes more time, and all of a sudden some other high priority process arrives, like **interrupt to avoid system crash**, the high priority process will have to wait, and hence in this case, the system will crash, just because of improper process scheduling.
- Not optimal Average Waiting Time.
- Resources utilization in parallel is not possible, which leads to **Convoy Effect**, and hence poor resource(CPU, I/O etc) utilization.

NIET
Greater Noida
Autonomous Institute
Get Future Ready

Shortest-Job-First (SJR) Scheduling

This algorithm associates with each process the length of the process's next CPU burst. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Shortest-Job-First (SJF) Scheduling

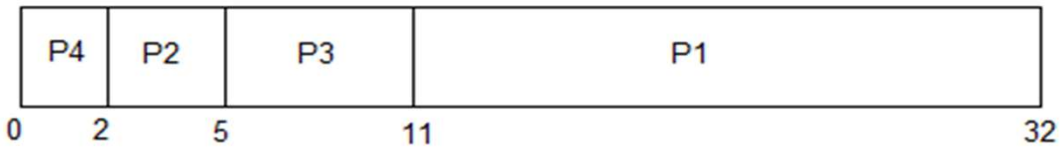
- Two schemes:
  - Non-preemptive – once CPU given to the process it cannot be preempted until completes its CPU burst.
  - Preemptive – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is know as the Shortest-Remaining-Time-First (SRTF).
- SJF is optimal – gives minimum average waiting time for a given set of processes.

Example of Non-Preemptive SJF

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2

In Shortest Job First Scheduling, the shortest Process is executed first.

Hence the GANTT chart will be following :



Now, the average waiting time will be = (0 + 2 + 5 + 11)/4 = 4.5 ms

NIET Greater Noida Autonomous Institute

Example of Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

SJF (preemptive)

Average waiting time = (9 + 1 + 0 +2)/4 = 3

Priority Scheduling

- Priority scheduling is a method of scheduling processes based on priority. In this method, the scheduler selects the tasks to work as per the priority.
- Priority scheduling also helps OS to involve priority assignments. The processes with higher priority should be carried out first, whereas jobs with equal priorities are carried out on a round-robin or FCFS basis.
- Priority can be decided based on memory requirements, time requirements, etc.

Priority Scheduling

Types of Priority Scheduling Algorithm:-

1.Preemptive Priority Scheduling: If the new process arrived at the ready queue has a higher priority than the currently running process, the CPU is preempted, which means the processing of the current process is stopped and the incoming new process with higher priority gets the CPU for its execution.

2.Non-Preemptive Priority Scheduling: In case of non-preemptive priority scheduling algorithm if a new process arrives with a higher priority than the current running process, the incoming process is put at the head of the ready queue, which means after the execution of the current process it will be processed.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Priority Scheduling (Non-preemptive) Example

Process	Arrival time	Burst time	Priority
P1	0 ms	5 ms	1
P2	1 ms	3 ms	2
P3	2 ms	8 ms	1
P4	3 ms	6 ms	3

NOTE: In this example, we are taking higer priority number as higher priority.

Gantt Chart

P1	P4	P2	P3
0ms 5ms	5ms 11ms	11ms 14ms	14ms 22ms

NIET Greater Noida
Autonomous Institute
Get Future Ready

Priority Scheduling (Non-preemptive) Example

Process	Waiting Time	Turnaround Time
P1	0ms	5ms
P2	10ms	13ms
P3	12ms	20ms
P4	2ms	8ms

Total waiting time: (0 + 10 + 12 + 2) = 24ms
Average waiting time: (24/4) = 6ms

Total turnaround time: (5 + 13 + 20 + 8) = 46ms
Average turnaround time: (46/4) = 11.5ms

NIET Greater Noida Autonomous Institute

Priority Scheduling

Advantages of priority scheduling (non-preemptive):

- Higher priority processes like system processes are executed first.

Disadvantages of priority scheduling (non-preemptive):

- It can lead to starvation if only higher priority process comes into the ready state.
- If the priorities of more two processes are the same, then we have to use some other scheduling algorithm.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Priority Scheduling

Problem with Priority Scheduling Algorithm

- In priority scheduling algorithm, the chances of **indefinite blocking** or **starvation**.
- A process is considered blocked when it is ready to run but has to wait for the CPU as some other process is running currently.
- But in case of priority scheduling if new higher priority processes keeps coming in the ready queue then the processes waiting in the ready queue with lower priority may have to wait for long durations before getting the CPU for execution.

NIET Greater Noida Autonomous Institute

Get Future Ready

Priority Scheduling

Using Aging Technique with Priority Scheduling

- To prevent starvation of any process, we can use the concept of **aging** where we keep on increasing the priority of low-priority process based on the its waiting time.
- For example, if we decide the aging factor to be **0.5** for each day of waiting, then if a process with priority **20**(which is comparatively low priority) comes in the ready queue. After one day of waiting, its priority is increased to **19.5** and so on. Doing so, we can ensure that no process will have to wait for indefinite time for getting CPU time for processing.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Round Robin

Round Robin(RR) scheduling algorithm is mainly designed for time-sharing systems. This algorithm is similar to FCFS scheduling, but in Round Robin(RR) scheduling, preemption is added which enables the system to switch between processes.

- A fixed time is allotted to each process, called a **quantum**, for execution.
- Once a process is executed for the given time period that process is preempted and another process executes for the given time period.
- Context switching is used to save states of preempted processes.
- This algorithm is simple and easy to implement
- It is important to note here that the length of time quantum is generally from 10 to 100 milliseconds in length.

Example of RR

Process	Arrival time	Burst time
P1	0 ms	10 ms
P2	0 ms	5 ms
P3	0 ms	8 ms

Gantt Chart

P1		P2		P3		P1		P2		P3	
1	2	2	4	4	6	6	8	8	10	10	12

P1		P2		P3		P1		P3		P1	
12	14	14	15	15	17	17	19	19	21	21	23

Example of RR

Process	Waiting Time	Turnaround Time
P1	13ms	23ms
P2	10ms	15ms
P3	13ms	21ms

Total waiting time: (13 + 10 + 13) = 36ms
Average waiting time: (36/3) = 12ms

Total turnaround time: (23 + 15 + 21) = 59ms
Average turnaround time: (59/3) = 19.66ms

NIET
Greater Noida
Autonomous Institute
Get Future Ready

Advantages and Disadvantages of RR

Advantages of round-robin:

- No starvation will be there in round-robin because every process will get chance for its execution.
- Used in time-sharing systems.

Disadvantages of round-robin:

- We have to perform a lot of context switching here, which will keep the CPU idle.

NIET Greater Noida
Autonomous Institute
Get Future Ready

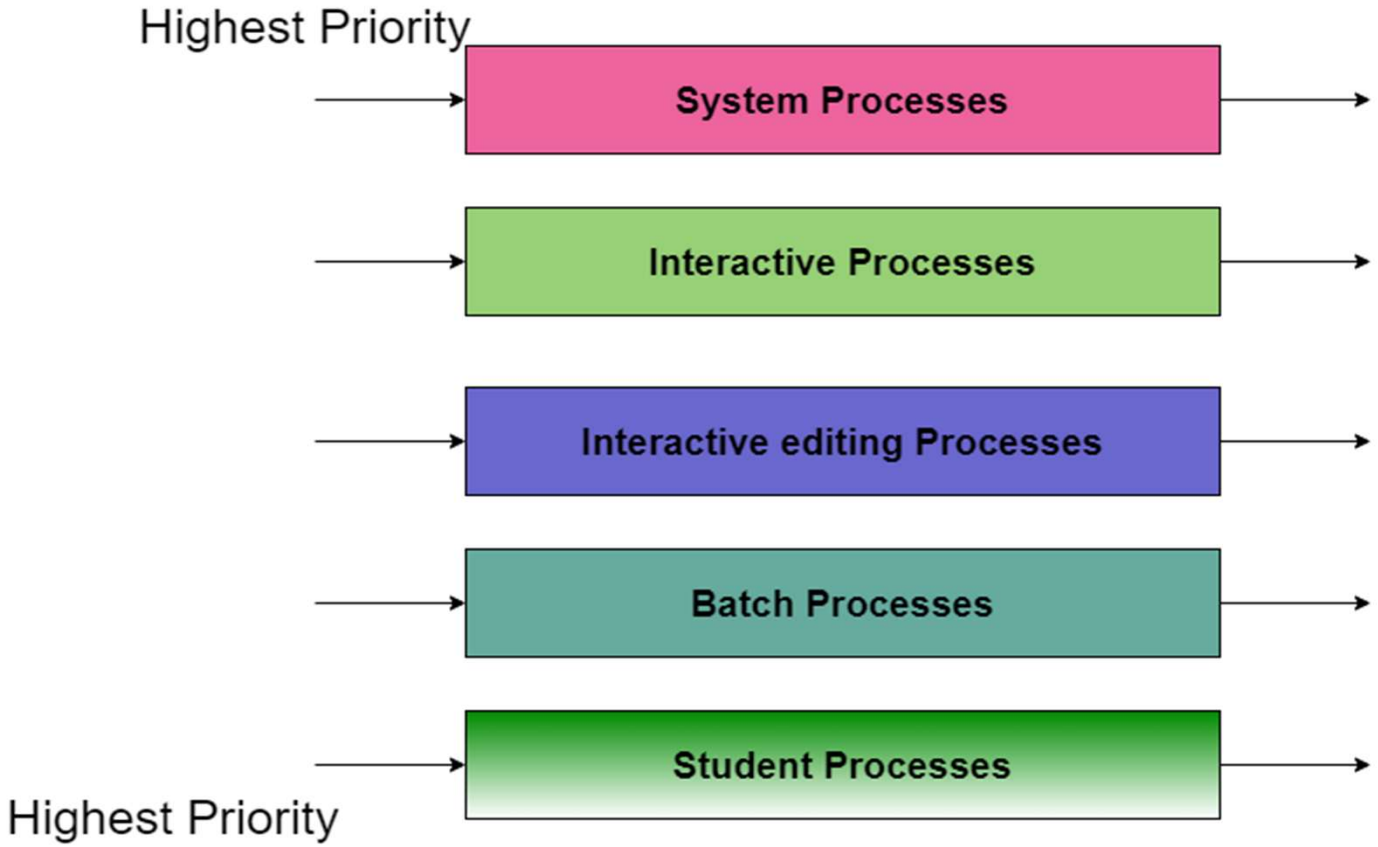
Multilevel Queue

- Ready queue is partitioned into separate queues: foreground (interactive) background (batch)
- Each queue has its own scheduling algorithm,

  foreground – RR
  background – FCFS
- Scheduling must be done between the queues.
  - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
  - 20% to background in FCFS

NIET Greater Noida
Autonomous Institute
Get Future Ready

Multilevel Queue Scheduling



NIET Greater Noida
Autonomous Institute
Get Future Ready

Multilevel Queue Scheduling

- **System Process** The Operating system itself has its own process to run and is termed as System Process.
- **Interactive Process** The Interactive Process is a process in which there should be the same kind of interaction (basically an online game).
- **Batch Processes** Batch processing is basically a technique in the Operating system that collects the programs and data together in the form of the **batch** before the **processing** starts.
- **Student Process** The system process always gets the highest priority while the student processes always get the lowest priority.

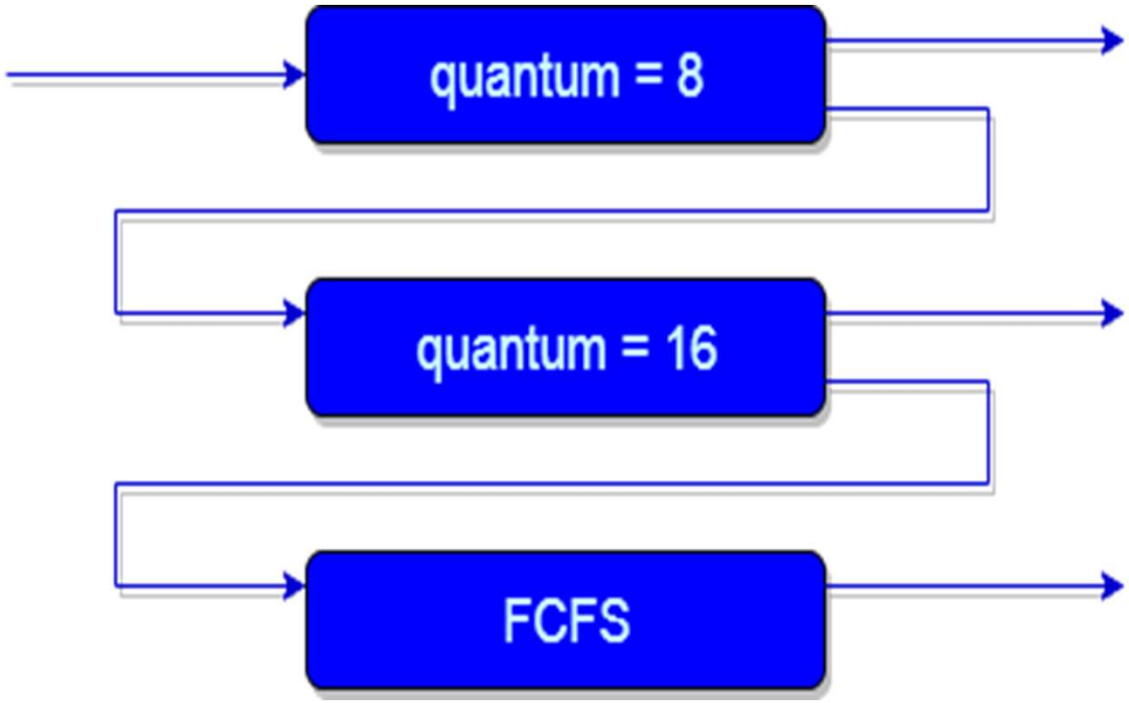
NIET Greater Noida Autonomous Institute

Multilevel Feedback Queue

In a multilevel queue-scheduling algorithm, processes are permanently assigned to a queue on entry to the system. Processes do not move between queues. In general, a multilevel feedback queue scheduler is defined by the following parameters:

- The number of queues.
- The scheduling algorithm for each queue.
- The method used to determine when to upgrade a process to a higher-priority queue.
- The method used to determine when to demote a process to a lower-priority queue.
- The method used to determine which queue a process will enter when that process needs service.

Multilevel Feedback Queue



NIET Greater Noida
Autonomous Institute
Get Future Ready

Example of Multilevel Feedback Queues

- Three queues:
  - Q_0 – time quantum 8 milliseconds
  - Q_1 – time quantum 16 milliseconds
  - Q_2 – FCFS
- Scheduling
  - A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1.
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2.

NIET Greater Noida Autonomous Institute

Get Future Ready

Need of Multilevel Feedback Queues

- This scheduling is more flexible than Multilevel queue scheduling.
- This algorithm helps in reducing the response time.
- In order to optimize the turnaround time, the SJF algorithm is needed which basically requires the running time of processes in order to schedule them. As we know that the running time of processes is not known in advance. Also, this scheduling mainly runs a process for a time quantum and after that, it can change the priority of the process if the process is long. Thus this scheduling algorithm mainly learns from the past behavior of the processes and then it can predict the future behavior of the processes.

NIET Greater Noida Autonomous Institute

Advantages of Multilevel Feedback Queues

Advantages of MFQS

- This is a flexible Scheduling Algorithm
- This scheduling algorithm allows different processes to move between different queues.
- In this algorithm, A process that waits too long in a lower priority queue may be moved to a higher priority queue which helps in preventing starvation.

NIET Greater Noida Autonomous Institute

Disadvantages of Multilevel Feedback Queues

Disadvantages of MFQS

- This algorithm is too complex.
- As processes are moving around different queues which leads to the production of more CPU overheads.
- In order to select the best scheduler this algorithm requires some other means to select the values.

NIET Greater Noida
Autonomous Institute
Get Future Ready

CPU Scheduling Algorithms (Preemptive & Non-Preemptive) Recap

Recap:

FCFS: Simple, non-preemptive.
SJF/SRTF: Based on burst time.
Priority: Based on priority levels (preemptive/non-preemptive).
Round Robin: Fixed time slice (preemptive).
Multilevel Queue: Multiple queues with fixed priority.
Multilevel Feedback: Queues allow movement for dynamic adjustment.

NIET Greater Noida
Autonomous Institute
Get Future Ready

CPU Scheduling Algorithms (Preemptive & Non-Preemptive) Recap

Scheduler with equal time slices for all tasks.

A) FCFS
B) SJF
C) Round Robin*
D) Priority

Algorithm selecting the process with shortest burst time.

A) FCFS
B) SJF*
C)Priority
D) Round Robin

Scheduler using dynamic feedback for process priority.

A) FCFS
B) SJF
C) Round Robin
D) Multilevel Feedback Queue*

Topic 7 (Objective & Prerequisite)

Processes and Threads

Objective:

Students will be able to differentiate between processes and threads, explain their relationships, and describe how threads enhance performance and resource sharing.

Prerequisite:

- Concept of multitasking and process memory space.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Thread

- A thread is a basic unit of CPU utilization
- Thread is an execution unit which consists of its own program counter, a stack, and a set of registers.
- Threads are also known as Lightweight processes
- Threads are popular way to improve application through parallelism.
- As each thread has its own independent resource for process execution, multpile processes can be executed parallely by increasing number of threads.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Process Vs Thread

Feature	Process	Thread
Definition	A process is an independent program in execution.	A thread is the smallest unit of execution within a process
Memory	Has its **own memory space** (code, data, stack).	Shares **memory** with other threads in the same process.
Communication	Inter-process communication (IPC) is **slower** and more complex.	Communication between threads is **faster** since they share memory.
Overhead	**More overhead** due to separate memory and resources.	**Less overhead**; lightweight compared to processes.
Creation time	**Slower** to create and manage.	**Faster** to create and manage
Failure impact	A process crash **does not affect** other processes.	A thread crash can **affect the entire process**.
Example	Running Chrome.exe and MS Word are separate processes.	Multiple tabs in Chrome may run as threads in the same process.
Summary	**Processes** are independent and isolated.	**Threads** are parts of a process that run concurrently and share resources.

Process Vs Thread (Recap)

Recap:

Threads share the same process memory.
Lighter than processes and useful for parallel tasks within the same process.

NIET Greater Noida Autonomous Institute

Get Future Ready

Process Vs Thread (MCQ)

Thread shares this resource with its parent process.
A) Program counter
B) CPU register
C) Memory space*
D) Instruction cycle
Thread is considered this part of a process.
A) Input
B) Instruction set
C) Execution unit*
D) Memory block
Threads use this to allow concurrent execution.
A) Different operating systems
B) Shared I/O devices
C) Shared code and memory*
D) Different languages

NIET Greater Noida
Autonomous Institute
Get Future Ready

Topic 8 (Objective & Prerequisite)

Thread States

Objective:

Students will identify and explain the different states of a thread and their transitions, similar to process state transitions.

Prerequisite

Familiarity with process states.

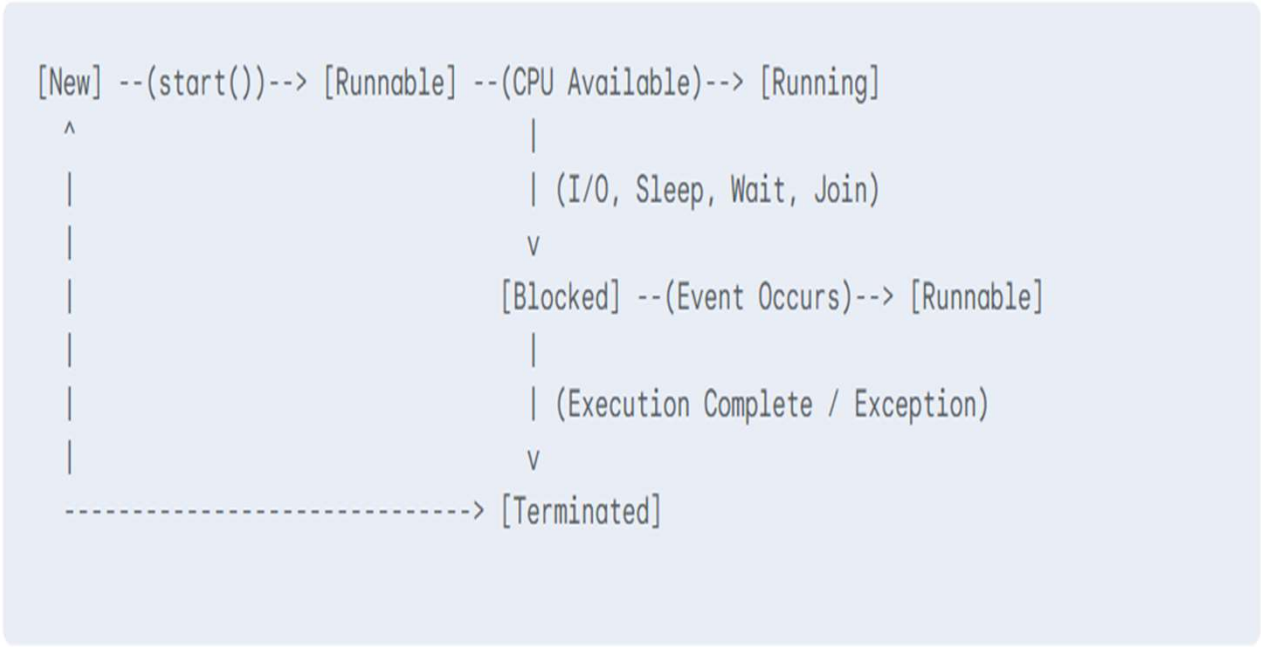
Thread State

Thread states

Thread states in list form:

1. New
2. Runnable
3. Running
4. Blocked
5. Waiting
6. Timed Waiting
7. Terminated

State Transition Diagram:



Thread State

- **New (or Born):**
  - A thread is in the new state when it has been created but has not yet started execution.
  - The operating system has allocated resources for it, but it's not yet ready to run.
  - *Example:* Thread thread = new Thread();
- **Runnable (or Ready):**
  - A thread is in the runnable state when it is ready to execute and waiting for the CPU.
  - It might be running or waiting for its turn to run, depending on the scheduler's policy and the availability of the CPU.
  - *Example:* thread.start(); (after this, it enters runnable state)

Thread State

Blocked (or Waiting):

- A thread enters the blocked state when it is temporarily inactive, waiting for some event to occur.
- Common reasons for blocking:

  I/O Operation: Waiting for input/output completion (e.g., reading from a file, network request).

  Resource Acquisition: Waiting for a lock or a resource to become available.

  Sleeping: Explicitly put to sleep for a specified duration.

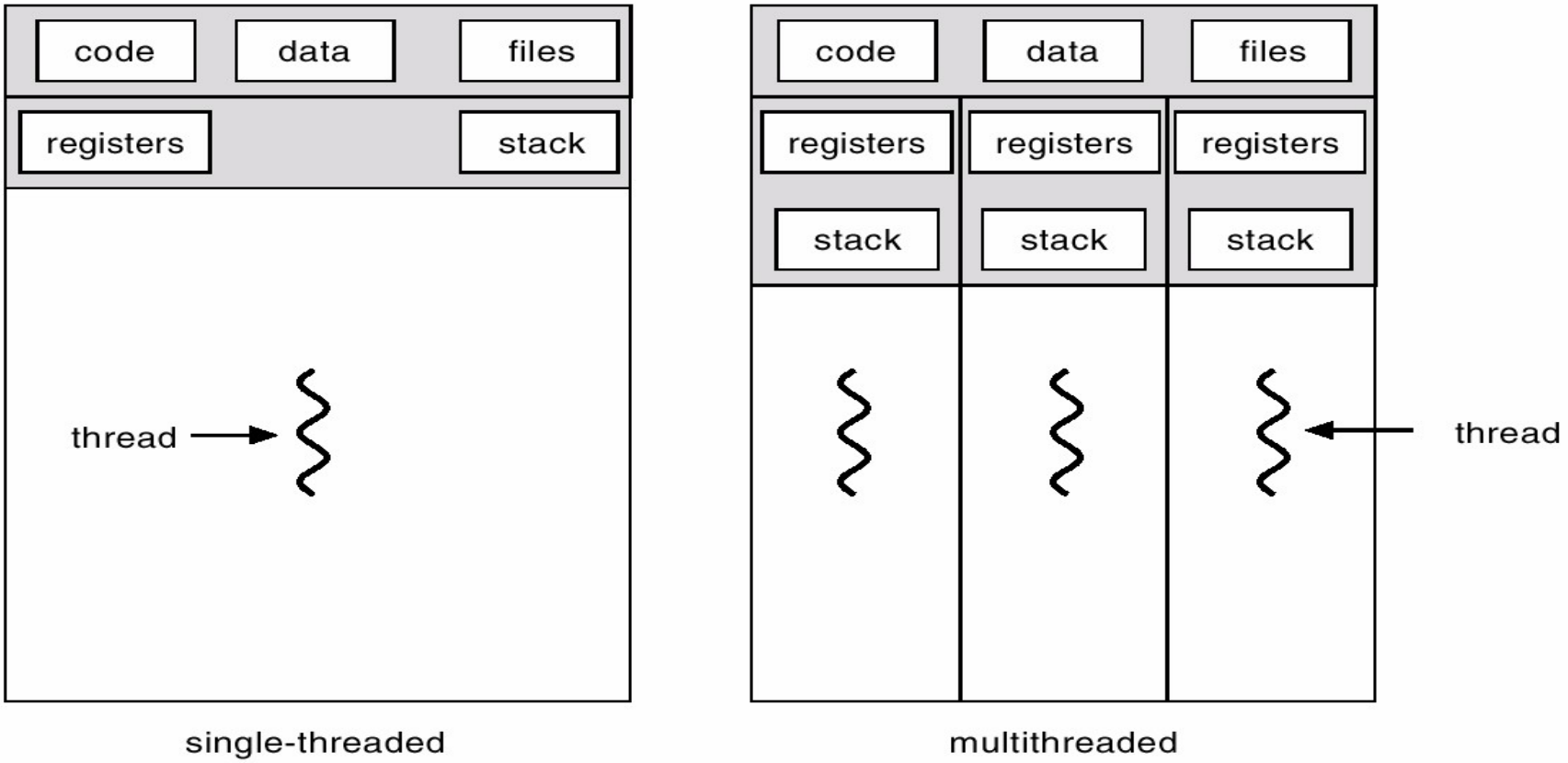
  Joining: Waiting for another thread to complete its execution.
- Once the event occurs, the thread moves back to the Runnable state.
- *Example:* thread.join();, Thread.sleep(1000);, synchronized (object) { object.wait(); }

Thread State

Terminated (or Dead):

- A thread is in the terminated state when it has completed its execution or has been abnormally terminated (e.g., due to an unhandled exception).
- Once a thread is terminated, it cannot be restarted.

Single and Multithreaded Processes



NIET Greater Noida
Autonomous Institute
Get Future Ready

Thread State (Recap)

Recap:

States: New, Runnable, Running, Waiting, Terminated (similar to process).

Managed by thread library or OS.

Thread State (MCQ)

State after thread completes execution.

A) Ready
B) New
C) Terminated*
D) Waiting

Thread moves to this state while waiting for I/O.

A) Terminated
B) Running
C) Waiting*
D) Ready

State assigned to thread after it is created.

A) Terminated
B) New*
C) Running
D) Blocked

Topic 9 (Objective & Prerequisite)

Benefits of Threads

Objective:

Students will recognize the advantages of using threads for concurrent programming and system performance improvement.

Prerequisite:

- Basic understanding of performance issues in programs.

NIET Greater Noida Autonomous Institute

Get Future Ready

Benefits of Multithreading

- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multiprocessor architectures

Benefits of Multithreading (Recap)

Recap:

- Efficient CPU usage, faster task switching, better responsiveness.
- Shared memory = easier communication.

NIET Greater Noida Autonomous Institute

Benefits of Multithreading (MCQ)

Threads increase system performance by enabling this.
A) Resource locking
B) Sequential execution
C) Parallelism*
D) I/O control

Thread-based applications offer faster results due to this.
A) Shared CPU
B) Increased complexity
C) Independent execution
D) Simultaneous task handling*

Threads improve responsiveness in this type of application.
A) Batch
B) Interactive*
C) Command-line
D) Embedded

Topic 10 (Objective & Prerequisite)

Types of Threads (User-level vs Kernel-level)

Objective:

Students will differentiate between user-level and kernel-level threads and understand their implementation, benefits, and limitations.

Prerequisite:

Knowledge of single-threaded vs multi-threaded programs.

Types of Thread

- Threads can be broadly categorized into two main types based on where their management occurs: User-Level Threads and Kernel-Level Threads.

1. User-Level Threads (ULTs)

2. Kernel-Level Threads (KLTs)

User-Level Threads (ULTs)

User Threads

- Thread management done by user-level threads library
- No need for kernel intervention
- Drawback : all may run in single process. If one blocks, all block.
- Examples
  - POSIX Pthreads
  - Mach C-threads
  - Solaris threads

Kernel Threads

Kernel Threads

- Supported by the Kernel
- Generally slower to create than user threads
- If one blocks another in the application can be run
- Can be scheduled on different CPUs in multiprocessor
- Examples
- Windows 95/98/NT/2000, Solaris, Tru64 UNIX, BeOS, Linux

NIET Greater Noida
Autonomous Institute
Get Future Ready

Types of Threads (Recap)

Recap:

User-level threads: Managed by user libraries.

Kernel-level threads: Managed by OS.

Each has pros/cons in terms of overhead and flexibility.

Types of Threads (Recap)

Thread managed by operating system.

A) Main thread

B) System call

C) Kernel thread*

D) Input thread

User-level threads depend on this for execution.

A) Hardware timer

B) User-level thread library*

C) BIOS

D) Assembly language

Topic 11 (Objective & Prerequisite)

Multithread Model

Objective:

Students will understand multithreaded execution models (Many-to-One, One-to-One, Many-to-Many) and their impact on performance and OS support.

Prerequisite

Understanding of threads and their interaction with OS.

NIET Greater Noida Autonomous Institute

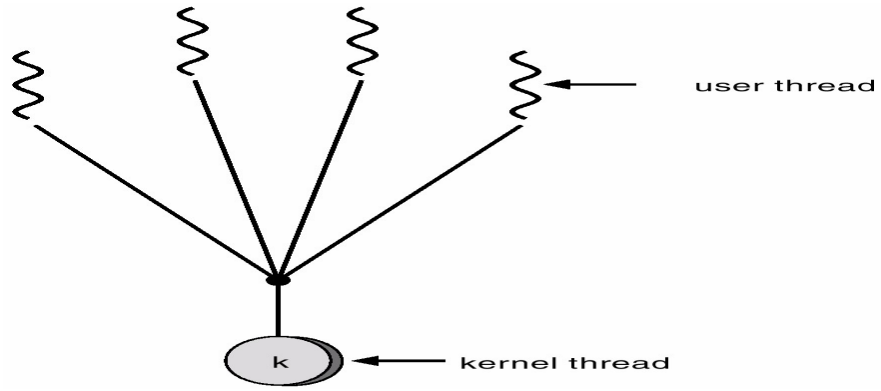
Get Future Ready

Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

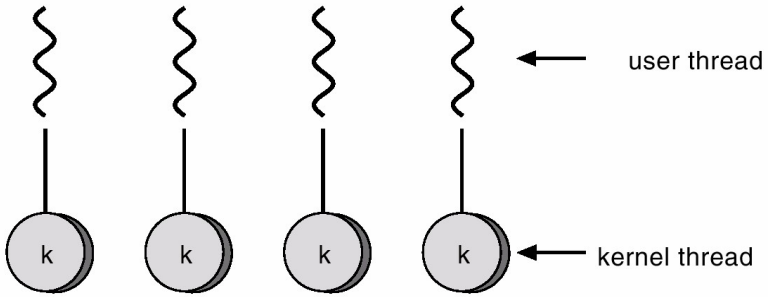
Many-to-One

- Many user-level threads mapped to single kernel thread.
- Used on systems that do not support kernel threads.



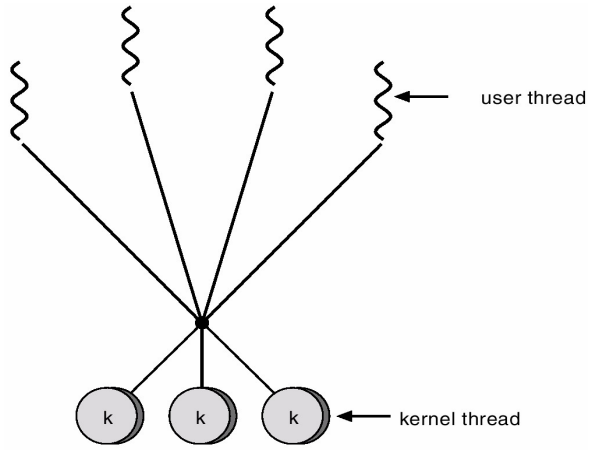
One-to-One

- Each user-level thread maps to kernel thread.
- Examples
- Windows 95/98/NT/2000
- OS/2



Many-to-Many Model

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**



NIET
Greater Noida
Autonomous Institute
Get Future Ready

Multithread Model (Recap)

Recap:

- Models: **Many-to-One**, **One-to-One**, **Many-to-Many**.
- Determines how user threads map to kernel threads.

Multithread Model (MCQ)

•Many-to-one model maps all threads to this.
A) Multiple CPUs
B) One process
C) Single kernel thread*
D) Multiple user IDs

•One-to-one model requires this for each user thread.
A) Shared I/O
B) Unique memory
C) Kernel thread*
D) CPU core

•Many-to-many model supports this flexibility.
A) Kernel-only threads
B) Dynamic thread mapping*
C) One thread per process
D) Static scheduling

Topic 12 (Objective & Prerequisite)

Hyper-Threading

Objective:

Students will learn about Intel's Hyper-Threading technology, how logical cores function, and how it affects parallelism and CPU utilization.

Prerequisite

Basic CPU architecture and instruction cycles.

NIET Greater Noida Autonomous Institute

Concept of Hyper-Threading

- **What is Hyper-Threading (HT)?**

➢ Hyper-Threading, developed by Intel, is a proprietary technology that improves parallelization of computations performed on x86 microprocessors.

➢ It is a form of **Simultaneous Multithreading (SMT)**, where a single physical CPU core is made to appear as two (or more) logical processors to the operating system.

➢ This means a single physical core can execute instructions from two different threads concurrently.

Concept of Hyper-Threading

- **How it Works:**

➢ Traditional CPU cores have multiple execution units (e.g., integer units, floating-point units, load/store units).

➢ Often, these execution units are underutilized by a single thread due to dependencies between instructions, cache misses, or waiting for data.

➢ Hyper-Threading allows two threads to share the core's execution resources. When one thread stalls (e.g., waiting for data from memory), the other thread can utilize the idle execution units.

➢ Each logical processor has its own architectural state (registers, program counter, interrupt controller), but they share most of the core's resources, including caches, execution units, and translation lookaside buffers (TLBs).

Concept of Hyper-Threading

- **Benefits:**

➢ **Improved Throughput:** Can increase the overall amount of work done by the CPU, especially when threads have different resource utilization patterns.

➢ **Better Resource Utilization:** Reduces idle time of execution units within a core.

➢ **Cost-Effective Parallelism:** Provides a degree of parallelism without requiring additional physical cores, making it a more economical solution for some workloads.

Concept of Hyper-Threading

- **Limitations and Considerations:**

➢ **Not True Parallelism:** It's not the same as having two physical cores. Two threads on a hyper-threaded core are still sharing resources, so performance gains are typically in the range of 15-30%, not 100%.

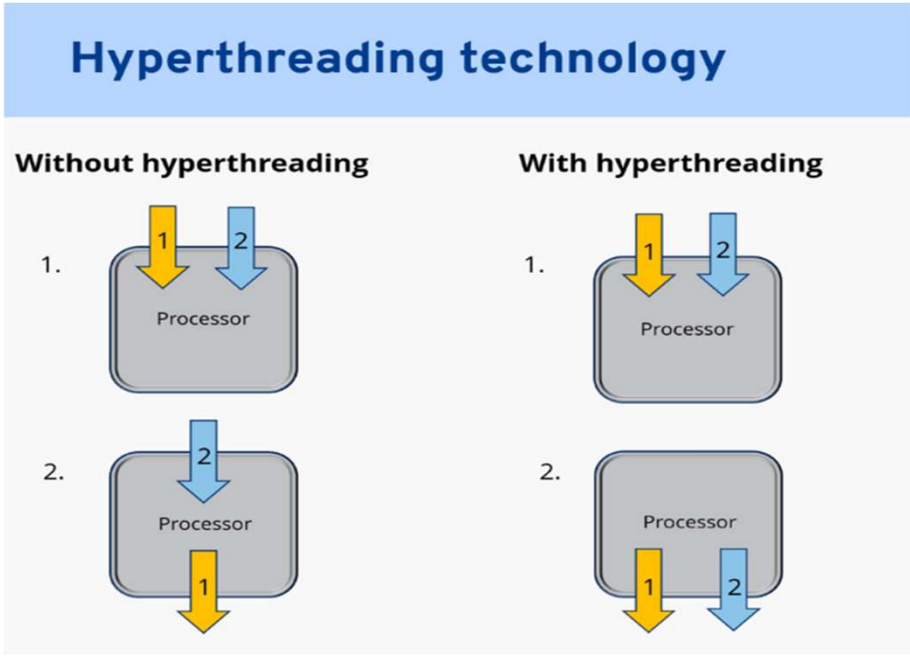
➢ **Contention:** If both threads are highly compute-bound and contend for the same shared resources (e.g., execution units, cache lines), performance can sometimes degrade or show minimal improvement.

➢ **Software Awareness:** Operating systems and schedulers need to be aware of HT to schedule threads efficiently. Modern OS schedulers are designed to prioritize scheduling threads on different physical cores before scheduling them on logical cores of the same physical core.

➢ **Security Concerns:** Some speculative execution vulnerabilities (like Spectre, Meltdown, L1TF) have been found to be exacerbated by Hyper-Threading, leading to recommendations to disable it in certain high-security environments.

Concept of Hyper-Threading

- **Use Cases:** Hyper-Threading is most beneficial for workloads that are not purely CPU-bound but involve a mix of computation and memory access, or when there are many threads waiting for I/O. It's widely used in desktop and server processors.



NIET Greater Noida Autonomous Institute

Concept of Hyper-Threading (Recap)

Recap:

- Intel technology: One physical core appears as two logical cores.
- Improves parallel execution and resource utilization.

NIET Greater Noida Autonomous Institute

Concept of Hyper-Threading (MCQ)

Hyper-threading improves performance by enabling this.
A) Single thread only
B) Instruction caching
C) Parallel thread execution*
D) Shared file system

Hyper-threading duplicates this component per core.
A) ALU
B) Thread control unit
C) Registers*
D) Instruction decoder

Hyper-threading simulates this per core.
A) One logical processor
B) Two logical processors*
C) Four physical processors
D) Multiple caches

Topic 13 (Objective & Prerequisite)

Case Study: CPU Scheduling in RTOS

Objective

Students will analyze and apply CPU scheduling techniques in real-time and embedded systems, emphasizing deadline-driven and deterministic behavior.

Prerequisite:

- Awareness of real-time systems and embedded programming basics.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

- **Introduction to Real-Time Embedded Systems (RTES) and RTOS:**
- **Real-Time Embedded Systems (RTES):**
  - Computer systems designed to perform specific functions within a strict time constraint.
  - Failure to meet deadlines can lead to system failure, catastrophic events, or safety hazards.
  - *Examples:* Automotive control systems (ABS, engine management), aerospace (avionics), industrial control, medical devices, robotics.
  - **Hard Real-Time Systems:** Missing a deadline is a catastrophic failure.
  - **Soft Real-Time Systems:** Missing a deadline is undesirable but not catastrophic; performance degrades.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

- **Real-Time Operating Systems (RTOS):**
  - An operating system specifically designed for real-time applications.
  - Key characteristics: **Determinism** (predictable timing behavior), **Predictability**, and **Low Latency**.
  - Prioritizes tasks based on their importance and deadlines, ensuring critical tasks complete on time.
  - *Examples:* FreeRTOS, VxWorks, QNX, RT-Linux, Zephyr.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

- **Challenges in CPU Scheduling for RTES:**

➢ CPU scheduling in RTES is fundamentally different from general-purpose operating systems due to the stringent timing requirements.

- **Deadline Guarantees:**
  - ➢ The primary challenge is to guarantee that critical tasks (often called "hard real-time tasks") meet their deadlines, even under peak load.
  - ➢ This requires schedulers that can analyze and predict worst-case execution times (WCET) and ensure schedulability.
- **Predictability vs. Throughput:**
  - ➢ General-purpose OS schedulers aim to maximize CPU utilization and throughput.
  - ➢ RTOS schedulers prioritize predictability and meeting deadlines, even if it means lower overall CPU utilization.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

- **Resource Contention and Priority Inversion:**
  - Tasks often share resources (e.g., shared memory, peripherals).
  - **Priority Inversion:** A high-priority task might get blocked by a lower-priority task that holds a required resource, leading to the high-priority task missing its deadline. RTOS must employ mechanisms like Priority Inheritance Protocol (PIP) or Priority Ceiling Protocol (PCP) to prevent this.
- **Jitter and Latency:**
  - **Jitter:** Variation in the execution time or response time of a task. RTES aims to minimize jitter for consistent behaviour.
- **Latency:** The time taken for a system to respond to an event. RTOS needs to ensure low interrupt and task switching latency.
- **Limited Resources:**
  - Embedded systems often have constrained memory, CPU power, and energy budgets. Schedulers must be lightweight and efficient.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

- **Common Scheduling Algorithms in RTOS:**

  RTOS schedulers are typically preemptive and priority-based.

1. Rate Monotonic Scheduling (RMS):

- **Type:** Static-priority, preemptive.
- **Principle:** Assigns higher priority to tasks with shorter periods (i.e., higher frequency).
- **Optimality:** Optimal among static-priority algorithms for a set of independent periodic tasks on a single processor.
- **Schedulability Test:** Utilizes the Liu & Layland bound. If the total CPU utilization of all tasks is below a certain threshold (e.g., ~69.3% for a large number of tasks), the system is schedulable.
- ***Pros:*** Simple to implement, good for periodic tasks.
- ***Cons:*** Not optimal for aperiodic tasks, requires prior knowledge of task periods

2. Earliest Deadline First (EDF):

- **Type:** Dynamic-priority, preemptive.
- **Principle:** Assigns higher priority to the task with the earliest absolute deadline.
- **Optimality:** Optimal among dynamic-priority algorithms for a set of independent periodic and aperiodic tasks on a single processor. If a set of tasks can be scheduled by any algorithm, it can be scheduled by EDF.
- **Schedulability Test:** A set of tasks is schedulable by EDF if their total CPU utilization is le100.
- *Pros:* Achieves full CPU utilization (if tasks are schedulable), flexible for mixed task types.
- *Cons:* More complex to implement (requires dynamic priority updates), can suffer from "domino effect" if a task misses its deadline, potentially causing others to miss theirs.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

3. Deadline Monotonic Scheduling (DMS):

- **Type:** Static-priority, preemptive.
- **Principle:** Assigns higher priority to tasks with shorter relative deadlines.
- Often performs better than RMS when task deadlines are not equal to their periods.

4. Fixed-Priority Preemptive Scheduling (FPPS):

- A general category that includes RMS and DMS. Tasks are assigned fixed priorities, and the highest-priority ready task always runs.

Other Considerations:

- **Resource Sharing Protocols:** PIP, PCP to manage shared resources and prevent priority inversion.
- **Interrupt Handling:** Minimizing interrupt latency and ensuring that Interrupt Service Routines (ISRs) are short and efficient.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

- **Implementation Considerations and Analysis:** Implementing CPU scheduling in RTES involves careful design and analysis to ensure real- time guarantees.

1. Task Analysis:

- **Identify Tasks:** Define all tasks in the system (periodic, sporadic, aperiodic).
- **Determine Parameters:** For each task, determine:
  - **Period (P):** For periodic tasks, how often it needs to run.
  - **Execution Time (C):** Worst-Case Execution Time (WCET) is crucial. This is often determined through static analysis, profiling, or measurement.
  - **Deadline (D):** The time by which a task must complete after its arrival.
  - **Priority:** Based on the chosen scheduling algorithm (e.g., shortest period for RMS, earliest deadline for EDF).

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

2. Schedulability Analysis:

- Before deployment, it's vital to mathematically prove that all critical tasks will meet their deadlines under all possible scenarios.
- **Utilization-Based Tests:** For RMS, check if

$$sum(C\_i/P\_i)$$
$$len(2^{1/n} - 1). \text{ For EDF, check if}$$
$$sum(C\_i/P\_i)$$
$$le1.$$

- **Response Time Analysis (RTA):** More precise and complex, calculates the worst-case response time for each task, considering preemption by higher-priority tasks. If the response time is less than or equal to the deadline, the task is schedulable.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

3. RTOS Configuration:

- **Tick Rate:** The frequency of the RTOS timer interrupt, which drives the scheduler. A higher tick rate provides finer granularity but increases overhead.
- **Task Stacks:** Each task requires its own stack space. Careful sizing is needed to avoid stack overflows while conserving memory.
- **Interrupt Management:** Proper handling of interrupts to ensure low latency for critical events and correct interaction with the scheduler.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

- **Example Scenario (Simplified RMS Analysis):**
- Consider a simple embedded system with two periodic tasks:
  - **Task 1:** $C_1=20ms$, $P_1=100ms$ (Deadline $D_1=100ms$)
  - **Task 2:** $C_2=30ms$, $P_2=150ms$ (Deadline $D_2=150ms$)

Step 1: Assign Priorities (RMS):

- Task 1 has a shorter period (100ms) than Task 2 (150ms).
- Therefore, Task 1 gets higher priority.

Step 2: Calculate CPU Utilization:

- $U_1=C_1/P_1=20/100=0.2$
- $U_2=C_2/P_2=30/150=0.2$
- Total Utilization $U=U_1+U_2=0.2+0.2=0.4$

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

Step 3: Apply Schedulability Test (Liu & Layland Bound for n=2):

- For $n = 2$ tasks, the bound is $2(2^{1/2} - 1)$
  $approx 0.828.$
- Since $U = 0.4$
  $le 0.828$, the system is schedulable using RMS. Both tasks will meet their deadlines.

5. Debugging and Monitoring:

Tools for tracing task execution, measuring response times, and identifying potential deadline misses are essential during development and testing.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS

Conclusion

Our case study on CPU scheduling in Real-Time Embedded Systems and RTOS highlighted the critical importance of predictability and deadline adherence in these specialized environments. Unlike general-purpose systems, RTOS prioritizes deterministic behaviour, employing sophisticated scheduling algorithms like Rate Monotonic and Earliest Deadline First to guarantee timely execution of critical tasks. Understanding these principles is vital for designing robust and reliable real-time applications.

NIET Greater Noida Autonomous Institute

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS (Recap)

Recap:

- Real-Time Operating Systems (RTOS) demand strict timing.
- Schedulers in RTOS are designed for deterministic behavior (e.g., Rate Monotonic, EDF).
- Emphasis on deadline handling and predictability.

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS (MCQ)

RTOS scheduling is designed to meet this requirement.

A) Maximum CPU usage
B) Predictable timing*
C) Multiple logins
D) User-level access

Algorithm most suitable for RTOS environments.

A) Round Robin
B) FCFS
C) Rate Monotonic*
D) SJF

RTOS uses priority scheduling for this reason.

A) Higher memory
B) Reduced I/O
C) Critical task handling*
D) Stack overflow

NIET Greater Noida Autonomous Institute

Faculty Video Links, Youtube & NPTEL Video Links and Online Courses Details

Youtube/other Video Links

- https://www.youtube.com/watch?v=_zOTMOubT1M
- https://www.youtube.com/playlist?list=PLmXKhU9FNesSFvj6gASuWmQd23Ul5omtD
- https://www.youtube.com/watch?v=x_UpLHXF9dU
- https://www.youtube.com/watch?v=cviEfwtdcEE
- https://nptel.ac.in/courses/106108101

NIET Greater Noida
Autonomous Institute
Get Future Ready

Daily Quiz

1. Which module gives control of the CPU to the process selected by the short-term scheduler?
   a) **dispatcher**
   b) interrupt
   c) scheduler
   d) none of the mentioned.

2. Which scheduling algorithm allocates the CPU first to the process that requests the CPU first?
   a) **first-come, first-served scheduling**
   b) shortest job scheduling
   c) priority scheduling
   d) none of the mentioned

NIET Greater Noida
Autonomous Institute
Get Future Ready

Daily Quiz

3. In priority scheduling algorithm _____
   a) **CPU is allocated to the process with highest priority**
   b) CPU is allocated to the process with lowest priority
   c) Equal priority processes can not be scheduled
   d) None of the mentioned

4. In priority scheduling algorithm, when a process arrives at the ready queue, its priority is compared with the priority of _____
   a) all process
   b) **currently running process**
   c) parent process
   d) init process

NIET Greater Noida
Autonomous Institute
Get Future Ready

Daily Quiz

5. Which algorithm is defined in Time quantum?
   a) shortest job scheduling algorithm
   b) **round robin scheduling algorithm**
   c) priority scheduling algorithm
   d) multilevel queue scheduling algorithm

6. . Process are classified into different groups in _____
   a) shortest job scheduling algorithm
   b) round robin scheduling algorithm
   c) priority scheduling algorithm
   d) **multilevel queue scheduling algorithm**

NIET
Greater Noida
Autonomous Institute
Get Future Ready

Weekly Assignment

1. Differentiate between preemptive and non-preemptive scheduling.
2. List out the necessary conditions for deadlock to occur.
3. Define PCB
4. List out the various scheduling criteria for CPU Scheduling.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Glossary Questions

1. A program in execution is referred to as a: _____
2. The state in which a process is waiting to be assigned to a processor is: _____
3. The memory area that stores the PCB of processes is managed by: _____
4. The transition of a process from running to waiting occurs due to: _____
5. The time spent by a process in the ready queue is known as: _____
6. The component that chooses the next process to execute is: _____
7. The module that gives control of the CPU to the selected process is: _____
8. The scheduler that handles job admission into the system is: _____
9. Context switching is performed by the: _____
10. The type of scheduling used in time-sharing systems is: _____

(Process Table, Waiting Time, Round Robin, I/O Request, Dispatcher, Long-Term Scheduler, Dispatcher Process, Ready, Scheduler)

NIET Greater Noida
Autonomous Institute
Get Future Ready

Old question paper

B.TECH.

Theory Examination (Semester-IV) 2015-16

OPERATING SYSTEM

Time : 3 Hours ***Max. Marks : 100***

Note: Attempt questions for all Sections as per directions.

Section-A

Attempt all parts of this section. Anser in brief. (2×10 = 20)

1. (a) What is SPOOLING?
   (b) Write a brief note on multiprocessor scheduling.
   (c) What is the need for Process Control Block (PCB)?
   (d) Define Multithreading.
   (e) Differentiate concurrent execution and parallel execution.
   (f) What do you understand by critical section?
   (g) How is a system call handled by the system?

(1) P.T.O.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Old question paper

(h) Draw process state transition diagram.

(i) What do you mean by Kernel?

(j) Define the multilevel feedback queues scheduling.

Section-B

2. Attempt any five questions from this section. (5×10 = 50)

(a) State the cause of thrashing and discuss its solution.

(b) What are the different techniques to remove fragmentation in case of multiprogramming with fixed partitions and variables partitions?

(c) Discuss the performance criteria for CPU Scheduling.

(d) Consider the following reference string 12342156212376321236. How many page faults will occur for:

(i) FIFO

NIET Greater Noida Autonomous Institute

Get Future Ready

Old question paper

(ii) LRU Page Replacement algorithm?

Assuming three and four frames in each case and frames are initially empty.

(e) Give the solution of Readers - Writers problem by using the concept of Semaphore.?

(2)

NIET Greater Noida
Autonomous Institute
Get Future Ready

Old question paper

(i) Bit vector (ii) Linked List

(iii) Grouping (iv) Counting

(g) Consider the processes, CPU burst time and Arrival time given below:

Processes	CPU burst time	Arrival time
PI	8	0
P2	4	1
P3	9	2
P4	5	3

Draw the Gantt chart and calculate the following by using **SRTF CPU** Scheduling Algorithm, (i) Average Waiting Time, (ii) Average Turn Around time.

(h) Consider the following snapshot of the system:-

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
PI	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				

NIET Greater Noida
Autonomous Institute
Get Future Ready

Old question paper

Answer the following questions using the banker's algorithm:-

(i) What is the content of the matrix **Need**?

(ii) Is the system in a **safe state**? If yes then find the **Safe sequence**.

(iii) If a request from process **PI** arrives for (0, 4, 2, 0) can the request be granted immediately?

Section-C

Attempt any two questions from this section. (2×15 = 30)

3. What is directory? Explain any two ways to implement the directory.

4. Suppose the moving head disk with 200 tracks is currently serving a request for track 143 and has just finished a request for track 125. If the queue of request is kept in FIFO order 86. 147. 91, 177. 94. 150. What is total head movement for the following scheduling:

(i) FCFS (ii) SSTF (iii) C-SCAN

NIET Greater Noida Autonomous Institute

Get Future Ready

Old question paper

5. Write short notes on:

(i) I/O Buffering

(ii) Sequential File

(iii) Indexed File

NIET Greater Noida Autonomous Institute

Expected Questions for University Exam

1. Draw the labeled process state transition diagram with describing the various process states.
2. Write a note on Deadlock Prevention.
3. Explain Multi Level Queue Scheduling.
4. Explain Short term, Middle term and Long term Schedulers.
5. Write down the steps of Deadlock Detection
6. Define PCB.
7. List out the various scheduling criteria for CPU Scheduling.
8. List out the necessary conditions for deadlock to occur.

Expected Questions for University Exam

9. Consider the following set of four processes, with the length of CPU burst time given in milliseconds

Process	Arrival Time	Burst Time
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

Draw Gannt chart and find average waiting time and response time using

- FCFS
- Round Robin (quantum=2)
- SJF and SRTF

NIET Greater Noida
Autonomous Institute
Get Future Ready

Recap of unit

- **Process Management Fundamentals:**
  - Understanding what a "Process" is.
  - Familiarity with the Process Transition Diagram (New, Ready, Running, Waiting, Terminated states).
  - Knowledge of the Process Control Block (PCB) and its role in storing process information.
  - Distinguishing between types of Schedulers: Long-Term, Mid-Term, and Short-Term.

NIET Greater Noida
Autonomous Institute
Get Future Ready

Recap of unit

- **CPU Scheduling Algorithms:**
  - Understanding the concepts of Pre-emptive and Non-Pre-emptive scheduling.
  - Detailed knowledge of various algorithms:
    - First-Come, First-Served (FCFS)
    - Shortest Job First (SJF)
    - Shortest Remaining Time First (SRTF)
    - Non-Pre-emptive Priority Scheduling
    - Pre-emptive Priority Scheduling
    - Round Robin (RR)
    - Multilevel Queue Scheduling
    - Multilevel Feedback Queue Scheduling

Recap of unit

Threads & Real-Time CPU Scheduling

Thread Concepts:

- Key differences between Processes and Threads.
- Understanding various Thread States (New, Runnable, Running, Blocked, Terminated).
- Benefits of using threads (Responsiveness, Resource Sharing, Economy, Scalability).
- Different Types of Threads (User-Level Threads, Kernel-Level Threads).
- Multithread Models (Many-to-One, One-to-One, Many-to-Many).
- The Concept of Hyper-Threading.

Recap of unit

Case Study: CPU Scheduling in Real-Time Embedded Systems and RTOS:

- Analysis of challenges in CPU scheduling for Real-Time Embedded Systems.
- Understanding Real-Time Operating Systems (RTOS) and their unique requirements.
- Implementation considerations for CPU scheduling in RTOS environments.